Mastering REST APIs and FastAPI

A comprehensive guide to building and consuming web APIs with Python

Made with GAMMA

What is an API?

Communication Bridge

An API (Application Programming Interface) allows two software applications to exchange data and communicate with each other seamlessly.

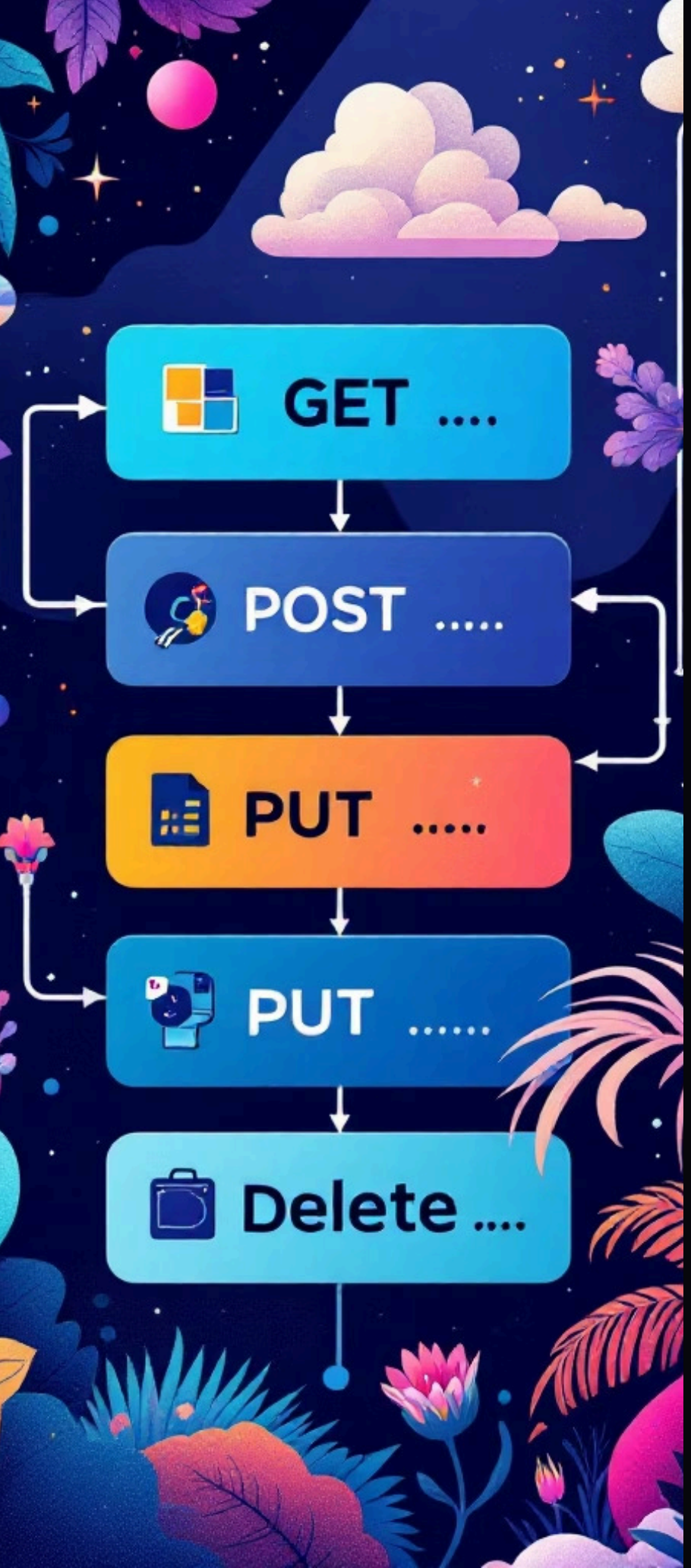
Real-World Example

Weather apps fetch real-time data from weather APIs. Instagram uses its API to share photos and user information across platforms.

Why APIs Matter

They enable data sharing between systems, connect different services, and help build scalable, modular applications efficiently.

Made with GAMMA



Understanding REST APIs

REST Principles

Representational State Transfer uses standard HTTP methods and works with URLs to request and send data in JSON format.

HTTP Methods

GET fetches data, **POST** creates new records, **PUT** updates existing data, and **DELETE** removes entries.

01

GET Request

Retrieve data from server

02

POST Request

Create new resource

03

PUT Request

Update existing data

04

DELETE Request

Remove data permanently

Made with GAMMA

Working with JSON Data

What is JSON?

JavaScript Object Notation is a lightweight data format used to send and receive data through APIs. It's easy to read and works across all programming languages.

Why JSON?

JSON is lightweight, human-readable, and language-independent, making it perfect for API communication and data exchange between systems.

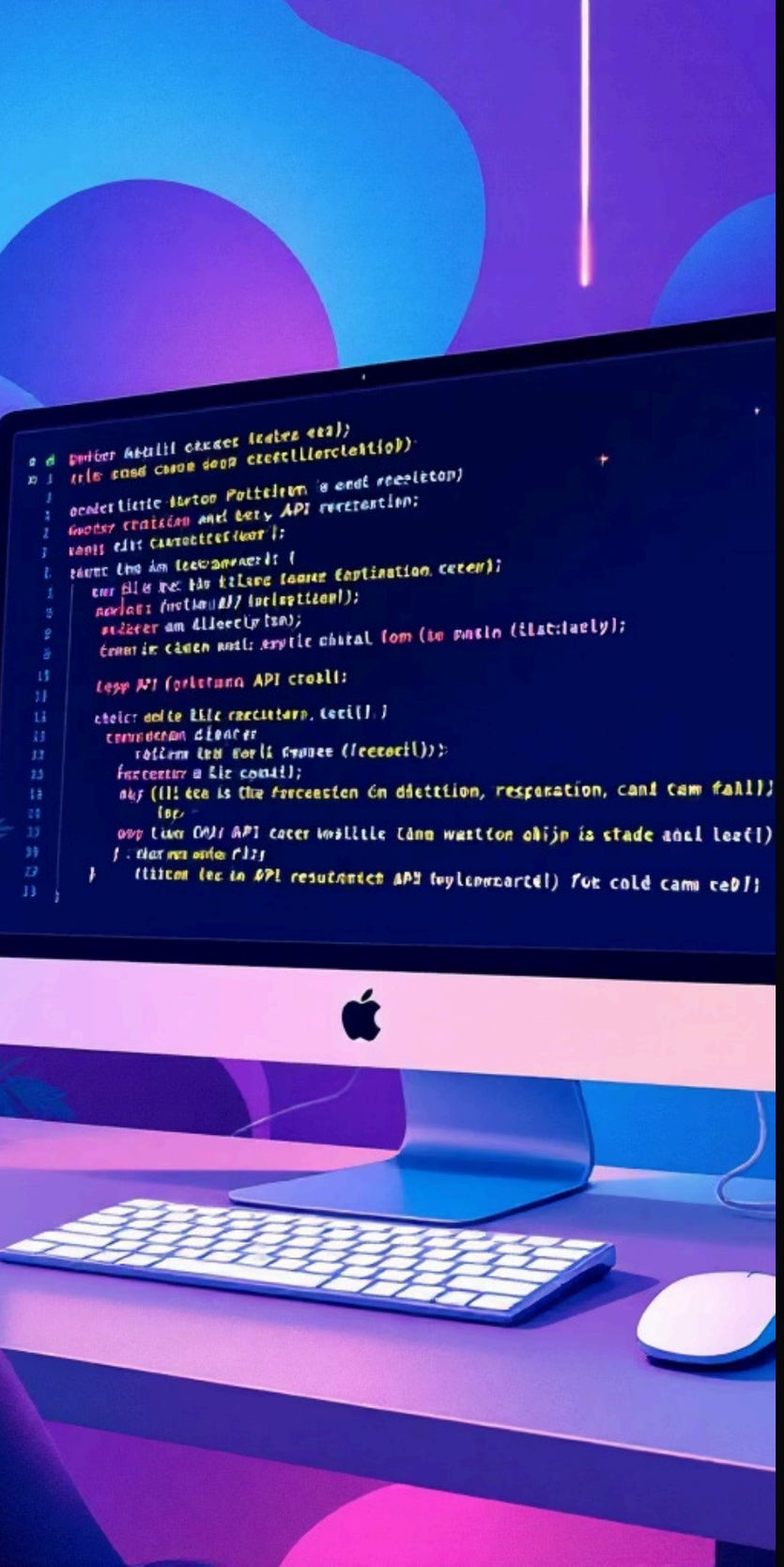
Example JSON Structure

```
{
  "name": "Gopika",
  "course": "Python NLP",
  "year": 3,
  "active": true
}
```

Key Features

- Key-value pairs for organized data
- Supports strings, numbers, booleans
- Can contain arrays and nested objects

Made with GAMMA



Consuming REST APIs with Python

Install requests Library

```
pip install requests
```

Send GET Request

```
import requests
url = "https://api.example.com/users"
response = requests.get(url)
```

Parse JSON Response

```
data = response.json()
print(data["name"])
```

Example: Fetch Wikipedia summary with GET request to https://en.wikipedia.org/api/rest_v1/page/summary/Python

Made with GAMMA

Parsing JSON and Saving Data

JSON Parsing

Convert JSON responses into Python dictionaries for easy data manipulation and access.

Save to File

Store API data permanently by writing JSON content to text or JSON files.

Parse JSON Example

```
response = requests.get(
  "https://api.agify.io?name=gopika"
)
data = response.json()
print(f"Name: {data['name']}")
print(f"Age: {data['age']}")
```

Save Data Example

```
with open("data.txt", "w") as file:
 file.write(data["extract"])
print("Saved successfully!")
```

Made with GAMMA

Introduction to FastAPI

What is FastAPI?

A modern, high-performance Python framework for building APIs quickly with automatic documentation and built-in data validation.

Key Features

Extremely fast processing, automatic Swagger documentation, powerful data validation, and seamless JSON handling out of the box.

Installation

```
pip install fastapi uvicorn
```

Made with GAMMA

Building Your First FastAPI App

Create main.py

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def home():
  return {
    "message": "Welcome to NLP API"
  }
```

Run the Server

```
uvicorn main:app --reload
```

Server starts at `http://127.0.0.1:8000`

Endpoint: GET /

Returns welcome message as JSON response when accessing the root URL.

Development Mode

--reload flag automatically restarts server when code changes, speeding up development.

Made with GAMMA

FastAPI with JSON Responses

Add Path Parameters

```
@app.get("/student/{name}")
def get_student(name: str):
 return {
  "student_name": name,
  "course": "Traditional NLP",
  "status": "Active"
 }
```

Test Your API

Visit `http://127.0.0.1:8000/student/Gopika` to see JSON response with dynamic path parameters.

Automatic Docs

FastAPI generates interactive documentation at `/docs` (Swagger) and `/redoc` (Redoc) endpoints automatically.

Swagger Benefits: Test endpoints directly in browser, send different parameters, view JSON responses, and explore API structure visually.

Made with GAMMA

Practice Tasks

Task 1: Course Info

Create GET /course endpoint returning course name and duration in JSON format.

Task 2: Square Calculator

Build /square/{number} endpoint that returns number and its square value.

Task 3: Mini Project

Combine everything: fetch Wikipedia data, save to file, create endpoint returning content as JSON.

> "Practice makes perfect! These hands-on tasks will help you master API development with FastAPI."

Made with GAMMA